

INTERVIEW DEFENSE DOCUMENT

DocIngestQA v0.2.0

Pre-indexing QA auditor for RAG document ingestion pipelines.

11 deterministic checks for missing pages, OCR noise, duplicates, encoding corruption, and poor split boundaries.

Metric	Value
Checks (v0.1)	8 deterministic checks
Checks (v0.2)	11 deterministic checks (+3 new)
Demo corpus	64 chunks, 10 documents
Demo status	FAIL (10 missing pages)
Check results	4 PASS, 6 WARN, 1 FAIL
PyPI	<code>pip install docingestqa</code>
Dependencies	Zero non-stdlib (jinja2 for HTML only)
Python support	3.10, 3.11, 3.12

Sidharth Kriplani | May 2026

Section 1 — Hard Interview Q&A

Q1 — `page_coverage` fires on any missing page. What if the manifest is wrong — a PDF that was only ever 6 pages is listed as 8?

That is a manifest authoring error, not a DocIngestQA bug. `page_coverage` is explicitly document-manifest-driven: it compares the pages present in the chunk set against the expected page count declared in the source manifest. If the manifest over-declares pages, the check fires a false positive. The design choice is to trust the manifest and report the discrepancy — the cost of a missed missing page in a RAG index is worse than the cost of an over-cautious FAIL that triggers human review. The recommendation in the output directs the user to inspect parser logs, which will confirm whether the pages were ever extracted. If a team has documents with variable page counts they should omit those sources from the manifest or dynamically generate it from their parser output.

Q2 — The `chunk_overlap` check uses pairwise Jaccard on consecutive chunks. What are the failure modes of this approach?

Three main failure modes. First, it is order-dependent: chunks from the same source must be sorted by page number for the consecutive-pair logic to work. Out-of-order chunks produce non-overlapping pairs even if a splitter created a 90% overlap window. The check sorts by page where available, but `page=None` chunks are excluded. Second, 4-gram Jaccard is insensitive to semantic overlap — two chunks covering the same concept in different words will score near zero. The check targets the mechanical overlap from sliding-window splitters, not semantic redundancy. Third, it is $O(n)$ per source (consecutive only), which means a splitter that creates overlapping non-adjacent chunks (e.g., overlapping windows with `stride=1`) will be underdetected. The threshold is calibrated for the most common case: a step size that is too small relative to the window size producing high Jaccard in adjacent pairs.

Q3 — Why is the fail threshold for `page_coverage` set at 'any missing page' (>0) when other checks like `metadata_completeness` only FAIL above 20%?

Because the failure modes are asymmetric. A missing page in the chunk set means a portion of the source document is completely unindexed — it will never be retrieved. This is a structural gap, not a quality degradation. One missing page in a legal contract or technical specification can make the RAG system answer questions about that section from hallucination rather than ground truth. In contrast, missing metadata (source or page on a chunk) is a quality signal — the chunk is still indexed and retrievable. The fail threshold asymmetry directly reflects the retrievability impact: structural absence (`page_coverage` FAIL at >0) vs. quality degradation (`metadata_completeness` FAIL at $>20\%$).

Q4 — `split_quality` detects mid-sentence starts and ends. How is a mid-sentence start detected?

The heuristic checks whether the first token of a chunk appears to be a continuation — specifically: no capitalisation where a sentence-starting word is expected, or the chunk starts with a lowercase continuation word ('and', 'but', 'or', 'which', 'that', 'however', 'therefore') without a preceding sentence. Mid-sentence ends are detected when the chunk ends without sentence-terminal punctuation (.?!) and the last token is not a list item or heading. Navigation fragments — TOC entries like 'Chapter 3 47' or bare page numbers — are caught by a separate regex pattern. The check is deliberately permissive: it fires WARN at 10% flagged and

FAIL at 30%, because many legitimate chunks start with proper nouns, numbers, or code that don't look like sentence-initial capitals. The goal is to surface systematic splitter misconfiguration, not flag every edge case.

Q5 — DocIngestQA has zero non-stdlib dependencies except jinja2 for HTML output. What did you sacrifice to achieve this?

Three things. First, no semantic similarity checks — detecting whether two chunks cover the same concept requires sentence embeddings (e.g., sentence-transformers), which pull in torch and transformers. The chunk_overlap check uses lexical n-gram Jaccard, which catches mechanical splitter overlap but misses semantic overlap from paraphrase. Second, no language detection — encoding_health catches mojibake patterns via regex, but cannot distinguish between a legitimately multilingual document and encoding corruption in an unknown script. Third, no PDF parsing — DocIngestQA audits already-extracted chunks. Adding a PDF parser would require pdfminer or pypdf, breaking the zero-heavy-deps guarantee. These are acknowledged limitations in the README under 'What DocIngestQA is not'. The design trade-off is: maximally deployable (pip install in any environment, no CUDA required) at the cost of semantic-layer coverage.

Q6 — A user reports that encoding_health is flagging legitimate French text (é, ç, à) as mojibake. Why, and how do you fix it?

The mojibake patterns in the check target specific double-encoding sequences — Ã© (UTF-8 'é' misread as Latin-1), Â© (copyright sign double-encoded), Ã (standalone Ã from broken UTF-8). These are unambiguous: no correctly encoded document has bare Ã followed by a combining character unless the UTF-8 bytes were decoded with the wrong codec. However, if a user's text extraction pipeline is correct and produces proper Unicode é (U+00E9), none of the mojibake regexes will match. If the false positive is occurring, it means the chunks were extracted with correct encoding and the text contains a sequence the regex accidentally matches — this should be reported as a bug. The fix is to tighten the regex to require the full surrogate pair pattern, not just the prefix characters. A workaround is to lower mojibake_warn_rate in AuditConfig to 0.0, which effectively disables the check for that corpus.

Q7 — The near_duplicate threshold uses 5-gram Jaccard at 0.85. Why 5-gram and not 3-gram or token-level?

The 5-gram choice balances specificity vs. sensitivity for the target failure mode. 3-gram Jaccard is too aggressive — it fires on any two chunks that discuss the same topic in similar language, including legitimate parallel sections in legal or technical documents. Token-level Jaccard (1-gram) is insensitive to word order and produces false positives on any two chunks from the same domain. 5-gram contiguous n-grams require exact word-sequence matches across five tokens, which is rare in non-duplicated text but common when a chunk appears verbatim in two places (copy-paste duplication, repeated boilerplate, or a splitter that emitted the same page twice). The 0.85 threshold allows for minor tokenisation differences. The exact vs. near-duplicate distinction is preserved: exact duplicates (SHA-1 hash) are always flagged regardless of threshold.

Q8 — What does DocIngestQA NOT detect that still causes RAG failures in production?

Five major failure modes are outside scope. First, retrieval relevance: a chunk can pass all 11 checks but still never be the top-retrieved result for the queries it should answer — that requires embedding-space

evaluation. Second, answer faithfulness: a retrieved chunk that contains the right fact can still produce a hallucinated answer if the LLM generates beyond the evidence. Third, cross-chunk logical coherence: two valid chunks can contradict each other without DocIngestQA detecting it (it has no cross-chunk semantic reasoning). Fourth, version staleness: a chunk can be syntactically perfect but describe a deprecated API version — there is no temporal freshness check. Fifth, table and figure extraction quality: structured data extracted as raw text by a PDF parser produces parseable but semantically useless chunks. DocIngestQA will not flag a table that was extracted as space-separated text because the text is valid UTF-8 and the correct length.

Q9 — The CLI exits with code 1 on FAIL. What is the intended use case for this in a pipeline?

The exit code 1 on FAIL is designed for CI/CD gating in RAG pipeline automation. The intended workflow is: document ingestion job runs (PDF extraction, chunking) → DocIngestQA runs as a quality gate → if exit code is 0 (PASS or WARN), the indexing job proceeds to upsert chunks into the vector database → if exit code is 1 (FAIL), the indexing job is blocked and an alert is raised for human review. WARN is not a blocking exit code by design: it signals quality degradation that may be acceptable depending on the corpus (e.g., a few short chunks in a large corpus) but should be monitored. This mirrors the same fail-fast design used in schema validation and data contract testing — the cost of indexing bad chunks is higher than the cost of a rejected pipeline run.

Q10 — If a team runs DocIngestQA on a 1-million-chunk corpus, what is the performance profile?

The primary bottleneck is the `duplicate_chunks` check (pairwise near-duplicate detection using 5-gram Jaccard). The current implementation does pairwise comparison per source, which is $O(n^2)$ in the worst case for a single source with many chunks. For a 1M-chunk corpus distributed across 10,000 sources, the worst case is a source with, say, 50,000 chunks — that is 1.25 billion pairs. In practice, the per-source pairwise comparison is bounded by the fail threshold: once 30 pairs are found, the check short-circuits. For corpora where source sizes are modest (e.g., max 1,000 chunks per document), performance is tractable. The mitigation roadmap includes MinHash LSH (v0.3) for approximate near-duplicate detection at scale, which reduces the comparison complexity to $O(n \log n)$. All other checks are $O(n)$ per chunk.

Section 2 — What DocIngestQA Does Not Do

Not a document parser. DocIngestQA audits chunks you already generated. It does not extract text from PDFs, Word docs, or HTML. Parsing quality is upstream of this tool.

Not a retrieval evaluator. It does not measure whether your chunks rank correctly for any query set. Retrieval quality requires an eval harness with query-document relevance labels.

Not a RAG correctness checker. It does not verify answer faithfulness or hallucination rate. LLM output quality is downstream of chunking quality but not fully determined by it.

Not a semantic coherence checker. Two chunks that contradict each other can both pass all 11 checks. Cross-chunk logical consistency requires embedding-level or LLM-based review.

Not an embedding quality checker. Embedding isotropy, cluster separation, and retrieval precision are separate concerns requiring embedding-space analysis.